Supplementary Material: Experimental Replication Guide

Framework: Multi-Agent RAG Framework for Autonomous Software Engineering (MA-GraphRAG-OSE)

Experimentation Repo: https://github.com/ahmedsalem84/PAP6/tree/main

1. Introduction

This supplementary document provides a comprehensive guide for researchers, reviewers, and practitioners to replicate the experiments presented in the main manuscript. The MA-GraphRAG-OSE framework is an interactive, closed-loop orchestration system that combines Large Language Models (LLMs), Graph Retrieval-Augmented Generation (GraphRAG), and Metaheuristic Optimization to automate and refine software engineering tasks.

This guide walks the user through the graphical user interface (GUI) of the experimental platform, explaining how to configure parameters, select datasets, assign LLMs, and monitor the iterative optimization process.

2. Usage and Benefits of the Experimental Platform

The experimental platform provides a user-friendly, unified environment to test and benchmark the framework. Its primary usages and benefits include:

- **Accessibility & Ease of Use:** A centralized dashboard eliminates the need for complex command-line executions. Users can select combinations of datasets, models, and optimizers via intuitive dropdowns and sliders.
- **Rapid Prototyping & Benchmarking:** Researchers can easily compare different metaheuristic algorithms (e.g., Sparrow Search vs. Genetic Algorithm) or LLM backends (e.g., Llama2 vs. Phi vs. Mistral) under identical experimental conditions.
- **Real-Time Monitoring:** The platform provides transparent, real-time logging of the agent swarm's execution, Knowledge Graph construction, and the optimization engine's inner workings.
- **Reproducibility:** By exposing exact hyperparameter controls (e.g., internal iterations, outer cycles, repetitions), the platform ensures that external reviewers and readers can independently verify the statistical claims and performance metrics reported in the study.

---

3. Step-by-Step Replication Guideline

Step 3.1: Accessing the Configuration Interface

Upon launching the application, the user is presented with the main dashboard. The left-hand sidebar contains the **Experiment Configuration** panel, which is divided into four main chronological steps.

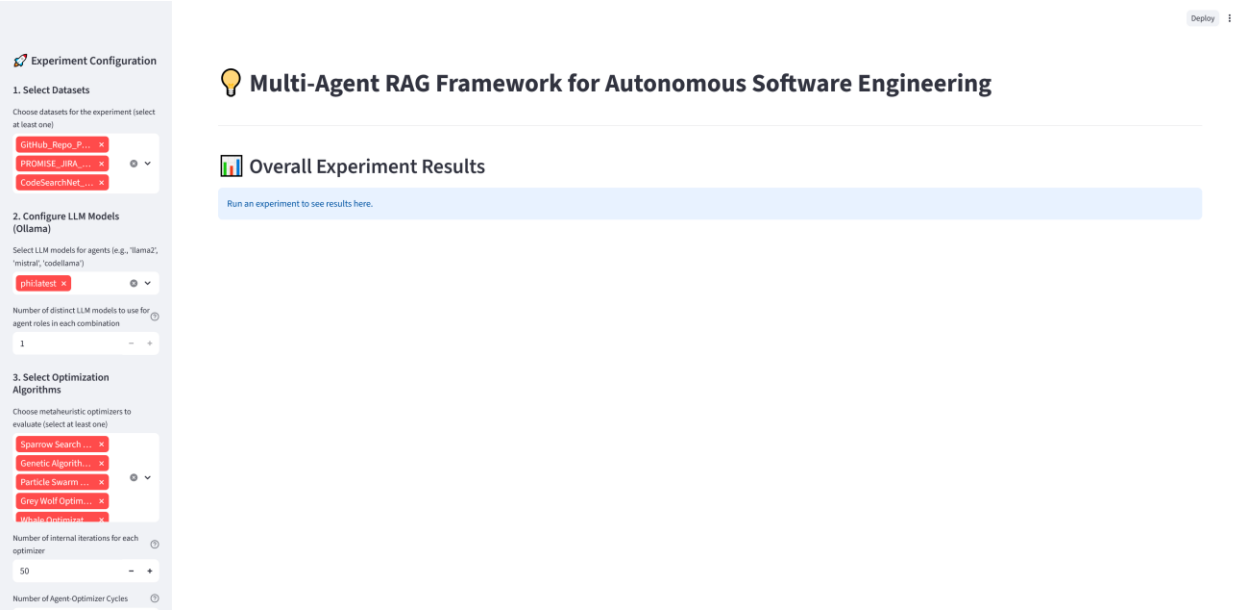


Figure S1*. The main dashboard and configuration sidebar of the MA-GraphRAG-OSE experimental platform.*

Step 3.2: Selecting Datasets and LLM Models

Using the sidebar, configure the data and intelligence backbone of the experiment:

1. **Select Datasets:** In the first section, select the target repositories. As shown in the configuration, you can select multiple benchmarks simultaneously (e.g., GitHub_Repo_Popular_Python, PROMISE_JIRA, and CodeSearchNet).
2. **Configure LLM Models (Ollama):** Select the local LLM backend used for the multi-agent swarm. In the example provided, phi:latest is selected. You can also define the *Number of distinct LLM models to use for agent roles*, which allows for testing heterogeneous agent assignments (e.g., assigning different models to the Testing vs. Architecture agents).

Step 3.3: Configuring the Optimization Engine

The optimization engine controls the closed-loop refinement of the software artifacts.

1. **Select Optimization Algorithms:** Choose one or more metaheuristics to benchmark. The platform supports multiple algorithms concurrently, such as *Sparrow Search, Genetic Algorithm, Particle Swarm, Grey Wolf Optimizer,* and *Whale Optimization*.
2. **Tuning Parameters:**
    - Set the **Number of internal iterations for each optimizer** (e.g., 50). This controls the search budget on the surrogate model.
    - Set the **Number of Agent-Optimizer Cycles** (e.g., 3). This determines how many full LLM generation passes the system will execute to refine the artifacts.

Step 3.4: Setting Repetitions and Initiating the Run

Scroll down the sidebar to finalize the execution parameters.

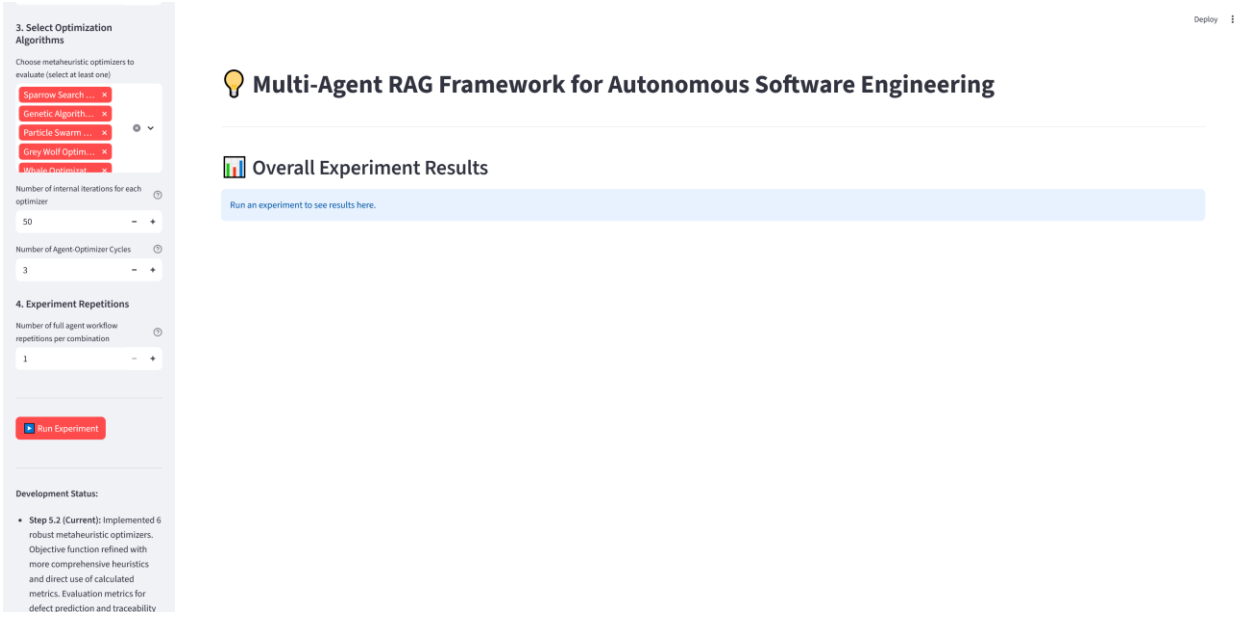


Figure S2*. Finalizing the experiment repetitions and initiating the execution.*

1. **Experiment Repetitions:** Set the *Number of full agent workflow repetitions per combination* (e.g., 1). For statistical significance testing (as done in the manuscript), this is typically set higher (e.g., 30).
2. **Initiate:** Click the red ▶ **Run Experiment** button to begin the closed-loop process. *(Note: The bottom of the sidebar also tracks the development/execution status, indicating which metrics and heuristics are actively loaded).*

Step 3.5: Monitoring Experiment Execution

Once the experiment is initiated, the main viewing area dynamically updates to show real-time progress logs.

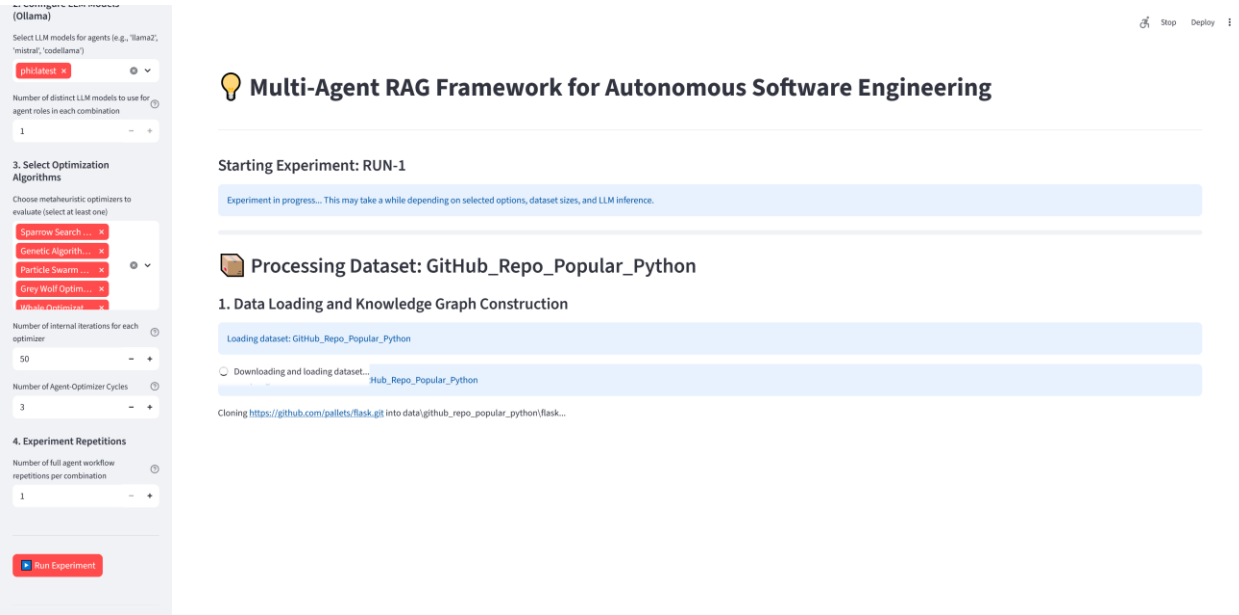


***Figure S3**. Real-time execution logs showing dataset processing and Knowledge Graph construction.*

What to Expect During Execution:

- **Run Initialization:** The system will display Starting Experiment: RUN-1 and a status banner indicating that the process is ongoing.
- **Data Processing:** The first major phase is *Data Loading and Knowledge Graph Construction*. As seen in the logs, the system will actively clone the selected repositories (e.g., cloning flask.git from GitHub) into the local data directory.
- **Subsequent Phases:** Following the cloning process, the platform will automatically:
    1. Parse the ASTs and commit histories to build the GraphRAG FAISS index.
    2. Sequentially trigger the LLM agents (Requirement, Architecture, Code, Test, Conflict, Traceability).
    3. Evaluate the generated artifacts against the objective function (Maintainability, Complexity, Test Pass Rate).
    4. Pass the metrics to the Metaheuristic optimizers to generate the translated prompt constraints (T(p)) for the next cycle.

4. Retrieving Outputs

Upon completion of all cycles and repetitions, the "Overall Experiment Results" section will populate with comparative tables, statistical metrics, and convergence trajectory graphs (similar to those presented in the manuscript's Results section). Raw logs and generated code artifacts are saved locally in the designated output directories for further analysis.